How It All Runs

AI-First Development — the technical walkthrough of every flow

Daily work • PR lifecycle • Promotion • Testing • Monitoring • Real-time bug fixing

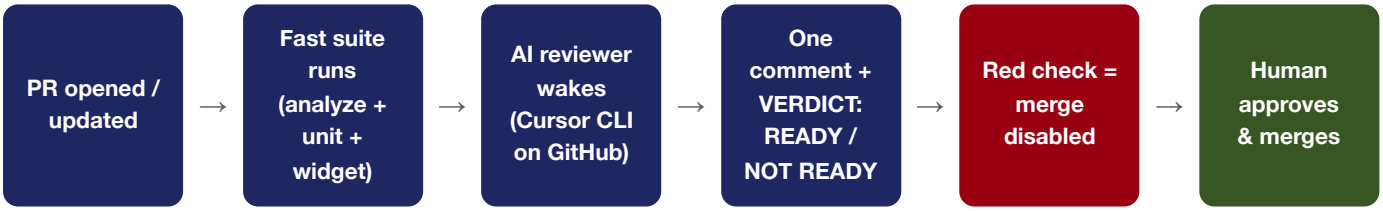
ArabGT — Mobile (Flutter) & Backend (Django) • Prepared by Omar Abed

1. Daily development — how a work session actually runs

1. A developer opens Cursor on either repo. **project.mdc** (the rules file) is injected automatically into every conversation — the AI already knows the stack, the feature anatomy, the navigation law, the red lines, and the behavior law: **not sure? ask — never guess.**
2. Every command the AI wants to run passes through the **hooks** first (`.cursor/hooks.json`, fail-closed). Production targets, secrets, destructive commands, force-push — rejected before execution, regardless of intent.
3. Recurring procedures are invoked with one word: `/sentry-sweep`, `/linear-ticket`, `/daily-report`, `/release-check`, `/parallel-tasks` — policy baked into each file.
4. For parallel work (Multitask, up to 8 agents): file ownership is mapped first — **no two agents ever touch the same file** — one git worktree + one branch per agent; the main checkout is never used by parallel agents.

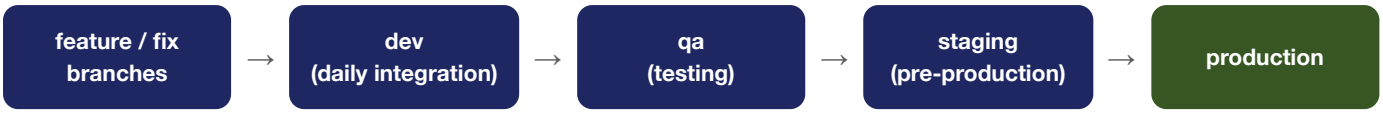
> **Why it holds**
>
> Everything above is plain files in the repo — versioned, PR-reviewed, delivered to every teammate on clone. Nothing depends on any individual's local settings.

2. Pull-request lifecycle — from open to merge



Mechanic	Detail
Reviewer identity	Our **pr-reviewer** agent (defined in `.cursor/agents/`) — explores the repo, applies project laws, checks tests exist. Read-only.
Incremental updates	A new push reviews **only the new diff** and re-checks earlier findings; the verdict always covers the whole PR.
NOT READY output	The comment ends with a ready **fix prompt** — paste into Cursor, apply, push; the check re-runs automatically.
Bugbot (hybrid)	Invoked by mention (`bugbot run`) only on staging → production promotions — an independent second review at the most critical gate.
Authority	The AI can open and review PRs. **It can never merge one.** Human approval is a hard branch-protection condition.

3. Branch & promotion flow



1. Sensitive branches accept **pull requests only** — direct pushes and force-pushes are blocked server-side at GitHub (applies to humans and AI, from any machine).
2. The **promotion gate** (a 2-second required check) enforces one law: **production accepts staging only.** Any other source fails instantly. dev and qa stay free for daily work.
3. `/release-check` gives a GO / NO-GO verdict before any promotion: full suite green, clean branch flow, and a diff scan proving no production/signing/secret files were touched.

4. Testing — the three layers and when each runs

Layer	What it proves	Environment
Unit	Pure logic behaves (thousands of checks per minute)	Bare runner — no device
Widget	Screens render correctly in memory (overflow, layout, states)	No simulator — fast
Journeys (Patrol)	A robot user walks the real journey — including **native** steps impossible for plain Flutter tests: permission dialogs, notification shade, WebView content	Booted simulator / real device

Schedule	What runs	Where & cost
Every PR	Full fast suite (layers 1–2)	Linux runner — cents
Nightly (Sun–Thu)	Full journeys on iOS simulator + Android emulator	GitHub runners (macOS minutes count 10x — schedule tuned accordingly)
Weekly	Same journeys on **real devices** — video + logs + report	Firebase Test Lab — inside the free daily quota at our volume
On demand	A button (or PR label): changed-feature journeys only, or the full app	Test Lab

Two principles

The AI writes a test once — every run afterwards is deterministic, cheap, and AI-free. And device farms receive only the **compiled app**: source code never leaves the company.

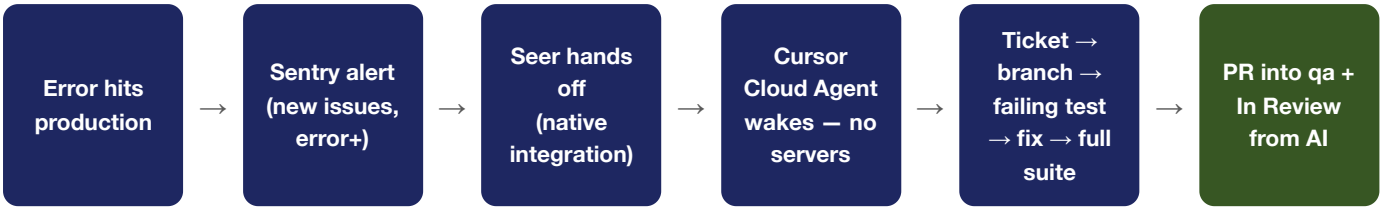
5. Daily monitoring rhythm — Sentry + PostHog

1. **07:00 — the sweep** (scheduled workflow): top unresolved Sentry issues → Linear cards. Dedupe first (existing card gets a comment, never a duplicate), **Sentry** label, problem-only body, and a **close-out section** (issue ID + link) at the bottom of every card.
2. **Morning report** (CI agent): yesterday's errors ranked (crashes first, then users affected) + which have Linear cards + notable patterns. Samples under ~10 events are labeled "*early signal — not a conclusion*".
3. **PostHog adds the behavior picture**: journeys, drop-offs, feature usage — merged into the same report: system health and user behavior in one view.

The standing law

Sentry stays READ-ONLY for the AI end to end. Creating tickets: yes. Resolving an issue: always a human — from the close-out link, after the fix is verified in production.

6. Real-time bug fixing — error to tested PR in minutes, zero infrastructure



1. A Sentry alert rule (new issues only, error level and above) lets **Seer** hand the issue straight to a **Cursor Cloud Agent** with the full error context. No relay, no servers — a native integration configured from Sentry settings.
2. The agent wakes on Cursor's cloud **with our repository laws applied** (rules live in the repo, so they follow it).
3. Linear: it creates or reuses the ticket (dedupe by issue ID), applies the policy body, moves it to **In Progress**.
4. The fix branches from **staging** — the code closest to what is actually broken — writes a **failing test first**, fixes the root cause, and runs the **full suite**.
5. One **PR into qa** (daily work on dev is never disturbed; the fix is carried back to dev too), a detailed comment on the ticket (root cause, files, test, PR link), and the status moves to **"In Review from AI"**.
6. Database, when needed (backend): most fixes need none — code + test fixtures. Otherwise a small **sanitized dump** over a secure link into a local PostgreSQL inside the agent's VM, destroyed with the run. **Staging is never touched, never exposed.**

Safety rail	Effect
One run per issue	No duplicate fix attempts for the same error
Never ship red	If the suite cannot go green, no PR is opened — findings land as a ticket comment instead
Human merge	The PR waits for AI review + human approval like any other; production still accepts staging only
Cost	Agent usage only — ≈ $0.5–3 per fix; no minutes, no infrastructure

Documented fallback

A GitHub-Actions bug-fix workflow (already committed in both repos) can run the same chain independently — kept dormant; switching back is one workflow activation.

7. Who can do what — the authority matrix

Action	AI	Human
Read code, run tests, local commits	Yes — freely	Yes
Push branches / open PRs	With approval (and CI paths by design)	Yes
Merge a PR	**Never**	Yes — after checks are green
Touch production targets, secrets, signing files	**Never — blocked by hooks**	Restricted by process
Create Linear tickets / comments	Yes — per policy (fixed project, dedupe)	Yes
Delete / restructure in Linear	**Never — blocked**	Yes
Read Sentry issues	Yes	Yes
Resolve / modify Sentry issues	**Never**	Yes — the close-out step

8. The control map — which file governs what

File (in both repos)	Governs
`.cursor/rules/project.mdc`	Project memory & laws — injected into every conversation
`.cursor/hooks.json` + `.cursor/hooks/*.sh`	Enforced guards on every command and MCP call — fail-closed
`.cursor/commands/*.md`	The five standard procedures (skills)
`.cursor/agents/*.md`	Specialist personas — reviewers (read-only), QA, PR judge
`.cursor/mcp.json`	Live connections: Sentry + Linear (per-user auth)
`.github/workflows/`	CI: PR review, promotion gate, scheduled tests, sweep — plus the dormant fallback pipeline

To switch it all on

GitHub Team upgrade ($4/user/mo) + required checks (10 admin minutes) + two service keys as secrets + Seer & the Cursor integration in Sentry settings. Everything else is already built, committed, and live-fire tested (guards: 21/21).

Universal laws across every flow: Sentry read-only • fixed destinations, no substitutes • unsure = ask • never ship red • merging is always human.